

Practical 3: Depth-First Search (DFS) Algorithm

Objective

To understand, implement, and analyze the Depth-First Search algorithm for graph traversal and various graph problems.

Theory

What is Depth-First Search?

Depth-First Search (DFS) is a graph traversal algorithm that explores as far as possible along each branch before backtracking. It uses a stack data structure (often implemented recursively using the call stack) to maintain the traversal order.

Algorithm Characteristics:

- **Time Complexity:** $O(V + E)$ where V is vertices and E is edges
- **Space Complexity:** $O(V)$ for the stack and visited array
- **Memory Usage:** Generally lower than BFS for wide graphs
- **Path Finding:** May not find the shortest path but explores deeply

DFS Algorithm Steps:

1. Start with a source vertex and mark it as visited
2. Add the source vertex to a stack
3. While the stack is not empty:
 - Pop a vertex from the top of the stack
 - If not visited, mark it as visited and add to result
 - For each unvisited neighbor of the current vertex:
 - Add the neighbor to the stack

Applications:

- Detecting cycles in graphs
- Topological sorting
- Finding strongly connected components
- Maze solving
- Parsing and syntax analysis
- Backtracking algorithms

DFS vs BFS Comparison:

Aspect	DFS	BFS
Data Structure	Stack	Queue
Memory Usage	Lower for wide graphs	Higher for wide graphs
Path Quality	May not find shortest	Finds shortest path
Completeness	Complete for finite graphs	Always complete
Applications	Cycle detection, topological sort	Shortest path, level traversal

DEPTH FIRST SEARCH

```
In [1]: import networkx as nx
import matplotlib.pyplot as plt
```

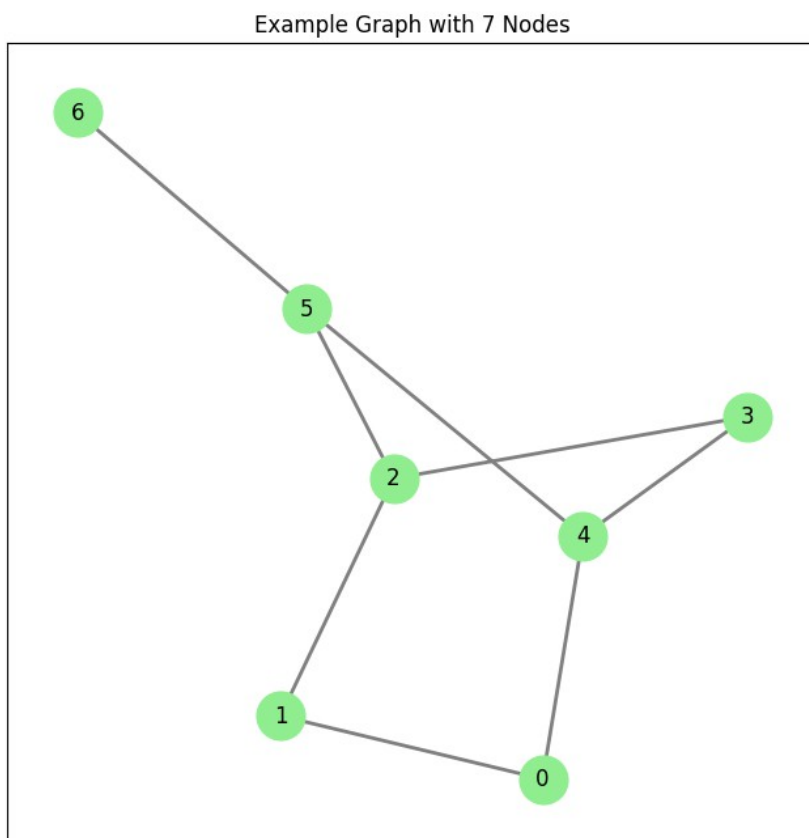
```
G = nx.Graph()
G.add_nodes_from(range(7))

edges = [
    (0, 1),
    (1, 2),
    (2, 3),
    (3, 4),
    (4, 5),
    (5, 6),
    (0, 4),
    (2, 5)
]

num_nodes = G.number_of_nodes()
G.add_edges_from(edges)
pos = nx.spring_layout(G)
```

```
In [2]: plt.figure(figsize=(8, 8))
nx.draw_networkx_nodes(G, pos, node_color='lightgreen', node_size=700)
nx.draw_networkx_edges(G, pos, width=2, edge_color='gray')
nx.draw_networkx_labels(G, pos, font_size=12, font_family='sans-serif')

# Set the title and display the plot.
plt.title("Example Graph with 7 Nodes")
plt.show()
```



In [3]: `import numpy as np`

```
class graph:
    def __init__(self , num_nodes , edges):
self.num_nodes = num_nodes
self.edges = edges
self.data = [[] for _ in range(num_nodes)]
        for n1, n2 in edges:
self.data[n1].append(n2)
self.data[n2].append(n1)

    def __repr__(self):
        return "\n".join(["{} : {}".format(n,x) for n,x in enumerate(self.data)])

    def __str__(self):
        return self.__repr__()

    def add_edge(self, n1, n2):
        if self.num_nodes is not None :
edges.append((n1,n2)) for n1, n2 in
edges:
        if n2 not in self.data[n1]:
self.data[n1].append(n2) if n1 not in
self.data[n2]:
self.data[n2].append(n1)
    def mat(self):
array = np.zeros(
self.num_nodes*self.num_nodes
).reshape(self.num_nodes,self.num_nodes) for
(x,y) in self.edges :
array[x][y]=1 return array

    def dfs(self , root):
stack = []
discovered = [False] * len(self.data) result =
[]

stack.append(root)

        while len(stack) > 0 :
current = stack.pop()
            if not discovered[current]:
discovered[current] = True
result.append(current)
                for node in self.data[current]:
                    if not discovered[node] :
stack.append(node)

        return result

    def check_connected_nodes(self):
result = self.dfs(self.edges[0][0])
        if len(result) == self.num_nodes:
            return True
        else :
            return False
```

```
In [4]: g1 = graph(num_nodes, edges)
        print(g1)
```

```
0 : [1, 4]
1 : [0, 2]
2 : [1, 3, 5]
3 : [2, 4]
4 : [3, 5, 0]
5 : [4, 6, 2]
6 : [5]
```

```
In [5]: g1.add_edge(2,6)
        print(g1)
```

```
0 : [1, 4]
1 : [0, 2]
2 : [1, 3, 5, 6]
3 : [2, 4]
4 : [3, 5, 0]
5 : [4, 6, 2]
6 : [5, 2]
```

```
In [6]: g1.dfs(3)
```

```
Out[6]: [3, 4, 0, 1, 2, 6, 5]
```

```
In [7]: g1.check_connected_nodes()
```

```
Out[7]: True
```